

TypeScript - Principiante

Tema 09: Clases

20 ejercicios progresivos, sin soluciones, para practicar clases, constructores, propiedades, metodos, estado interno, instancias y colaboracion entre objetos.

Ejercicio 1 - Persona

Crea una clase Person con name y age. Inicializa ambas propiedades mediante el constructor.

Requisitos

- Crea al menos dos instancias.
- Agrega un metodo describe que devuelva una descripcion.

Ejemplo de ejecucion

Ana tiene 28 anos

Objetivo

Practicar clases, constructores e instancias.

Pista

El constructor recibe los datos necesarios para crear el objeto.

Ejercicio 2 - Producto

Crea una clase Product con id, name y price.

Requisitos

- Inicializa las propiedades en el constructor.
- Agrega un metodo para mostrar informacion.

Ejemplo de ejecucion

1 - Teclado - \$850

Objetivo

Reforzar propiedades y metodos de instancia.

Pista

Cada instancia conserva sus propios valores.

Ejercicio 3 - Rectangulo

Crea una clase Rectangle con width y height.

Requisitos

- Agrega metodos getArea y getPerimeter.
- Ambos devuelven number.

Ejemplo de ejecucion

Area: 50 | Perimetro: 30

Objetivo

Practicar metodos que calculan datos.

Pista

Los metodos pueden utilizar propiedades de la instancia.

Ejercicio 4 - Contador

Crea una clase Counter con un valor inicial de 0.

Requisitos

- Agrega increment, decrement y getValue.
- Crea una instancia y realiza varias operaciones.

Ejemplo de ejecucion

Contador: 3

Objetivo

Practicar modificacion del estado de una instancia.

Pista

Cada metodo puede modificar una propiedad del objeto.

Ejercicio 5 - Cuenta bancaria

Crea una clase BankAccount con owner y balance.

Requisitos

- Agrega deposit y withdraw.
- No permitas cantidades invalidas.
- No permitas retirar mas que el saldo.

Ejemplo de ejecucion

Saldo final: \$1800

Objetivo

Practicar reglas dentro de metodos.

Pista

La clase puede proteger la consistencia de sus propios datos.

Ejercicio 6 - Estudiante

Crea una clase Student con name y un array de calificaciones.

Requisitos

- Agrega addGrade.
- Agrega getAverage.
- No permitas calificaciones fuera del rango definido.

Ejemplo de ejecucion

Promedio: 8.7

Objetivo

Combinar clases con arrays.

Pista

Las calificaciones forman parte del estado del estudiante.

Ejercicio 7 - Carrito

Crea Product y ShoppingCart. El carrito debe almacenar productos.

Requisitos

- Permite agregar productos.
- Permite listar productos.
- Calcula el total.

Ejemplo de ejecucion

Productos: 3 | Total: \$4700

Objetivo

Practicar colaboracion entre clases.

Pista

Una clase puede contener instancias de otra.

Ejercicio 8 - Tarea

Crea una clase Task con title y completed.

Requisitos

- completed inicia en false.
- Agrega complete y reopen.
- Agrega un metodo para mostrar el estado.

Ejemplo de ejecucion

Estudiar TypeScript - Completada

Objetivo

Practicar cambios de estado mediante metodos.

Pista

Evita modificar el estado desde codigo duplicado.

Ejercicio 9 - Libro

Crea una clase Book con title, author y available.

Requisitos

- Agrega borrow y returnBook.
- No permitas prestar un libro ya prestado.

Ejemplo de ejecucion

Libro prestado correctamente

Objetivo

Modelar comportamiento real mediante clases.

Pista

El metodo debe consultar el estado antes de cambiarlo.

Ejercicio 10 - Biblioteca

Crea una clase Library que administre una coleccion de Book.

Requisitos

- Agrega libros.
- Busca por titulo.
- Lista disponibles.
- Permite prestar mediante la biblioteca.

Ejemplo de ejecucion

Libros disponibles: 4

Objetivo

Practicar clases administradoras de colecciones.

Pista

Library puede delegar parte del comportamiento a Book.

Ejercicio 11 - Empleado

Crea Employee con id, name y salary.

Requisitos

- Agrega increaseSalary con un porcentaje.
- Rechaza porcentajes invalidos.
- Agrega getAnnualSalary.

Ejemplo de ejecucion

Salario anual: \$240000

Objetivo

Practicar logica de negocio dentro de clases.

Pista

Los calculos pueden exponerse mediante metodos.

Ejercicio 12 - Inventario

Crea InventoryItem con producto, stock y precio.

Requisitos

- Agrega addStock y removeStock.
- No permitas stock negativo.
- Calcula el valor del inventario del item.

Ejemplo de ejecucion

Stock: 10 | Valor: \$8500

Objetivo

Practicar invariantes de objetos.

Pista

Valida antes de modificar el estado.

Ejercicio 13 - Pedido

Crea OrderItem y Order.

Requisitos

- Order almacena OrderItem[].
- Permite agregar items.
- Calcula subtotal por item y total del pedido.

Ejemplo de ejecucion

Pedido 1001 - Total: \$2100

Objetivo

Modelar relaciones entre clases.

Pista

Divide responsabilidades entre Order y OrderItem.

Ejercicio 14 - Agenda

Crea Contact y ContactBook.

Requisitos

- Permite agregar contactos.
- Buscar por nombre.
- Eliminar por id.
- Listar todos.

Ejemplo de ejecucion

Contactos: 5

Objetivo

Practicar CRUD en memoria usando clases.

Pista

ContactBook administra una coleccion interna.

Ejercicio 15 - Temporizador conceptual

Crea una clase Timer con seconds.

Requisitos

- Permite avanzar segundos.
- Permite resetear.
- Devuelve minutos y segundos formateados.
- No necesitas usar temporizadores reales.

Ejemplo de ejecucion

02:15

Objetivo

Practicar encapsulacion de calculos y estado.

Pista

Trabaja solamente con operaciones sincronas.

Ejercicio 16 - Factura

Crea InvoiceItem e Invoice.

Requisitos

- Cada item tiene description, price y quantity.
- Invoice calcula subtotal, impuesto y total.
- La tasa de impuesto se recibe al crear la factura.

Ejemplo de ejecucion

Subtotal: \$2000 | Impuesto: \$320 | Total: \$2320

Objetivo

Practicar composicion y calculos entre objetos.

Pista

Invoice debe calcular a partir de sus items.

Ejercicio 17 - Usuario y sesion

Crea User y Session.

Requisitos

- Session recibe un User.
- Controla si la sesion esta activa.
- Permite login y logout.
- Evita operaciones inconsistentes.

Ejemplo de ejecucion

Usuario: Ana | Sesion: activa

Objetivo

Practicar objetos relacionados y estados.

Pista

La sesion puede conservar una referencia al usuario.

Ejercicio 18 - Repositorio en memoria

Crea UserRepository que almacene usuarios internamente.

Requisitos

- Implementa getAll, getById, add, update y delete.
- Evita ids duplicados.
- Maneja registros inexistentes.

Ejemplo de ejecucion

Usuarios registrados: 4

Objetivo

Practicar clases como servicios de datos.

Pista

Centraliza las operaciones sobre la coleccion.

Ejercicio 19 - Sistema de gastos

Crea Expense y ExpenseManager.

Requisitos

- Expense contiene id, concept, category y amount.
- ExpenseManager permite agregar, listar, buscar y eliminar.
- Calcula total general y total por categoria.
- Rechaza cantidades invalidas.

Ejemplo de ejecucion

Gastos: 5 | Total: \$3250

Objetivo

Integrar clases y colecciones.

Pista

Separa la representacion del gasto de la administracion de gastos.

Ejercicio 20 - Sistema de pedidos

Construye Customer, Product, OrderItem, Order y OrderService.

Requisitos

- Customer representa al cliente.
- Product contiene id, name, price y stock.
- OrderItem relaciona producto y cantidad.
- Order calcula su total.
- OrderService crea y administra pedidos.
- Al crear un pedido valida stock.
- Descuenta stock cuando corresponda.
- Permite buscar pedidos por cliente y estado.
- No uses any.

Ejemplo de ejecucion

Pedido 1001 creado

Cliente: Laura

Total: \$4800

Estado: pending

Objetivo

Integrar varias clases, responsabilidades, colecciones y reglas de negocio.

Pista

Define claramente que responsabilidad pertenece a cada clase.